



浙江大學
ZHEJIANG UNIVERSITY

边缘计算开发实践课程报告

团队大作业报告

组长 王颂凯

组员 李维扬、吕嘉鑫、王舜玺

院所 信息与电子工程学院

2025 年 11 月 8 日

目录

1	创意介绍	3
2	设计目标	3
3	实验过程	3
3.1	数据采集与模型训练	4
3.2	BLE 设备调试	5
3.3	MQTT 连接调试	6
3.4	应急方案与最终实现	7
4	扩展性与后续开发	9
5	反思与总结	10

1 创意介绍

本项目的设计灵感来自谍战片中经典的“脚步动作传讯”场景：特工执行任务时，无需语言交流，也不必依赖电台、密码本等复杂设备，只需通过预设的脚步动作——比如“三短一长”的轻踩节奏、特定方位的踏步幅度，就能向同伴精准传递“安全”“危险”“集合”等关键信息。我们认为这种功能可以概括为动作识别与无线通信的结合，一般利用穿戴式嵌入式设备可以实现。如果能够成功复现，在实际生活中也有相当多实用的应用场景：

- 在驾驶场景中，用手势（如“顺时针画圈调音量”）或脚部动作进行驾驶之外功能的操作，不需要驾驶员分心去找对应的按钮，提升操作安全性。
- 为行动不便人群设计的智能系统，如通过头部转动（“向左偏头 = 接电话”“向右偏头 = 挂电话”）控制智能家电与通讯设备。
- VR 游戏中，玩家的穿戴式设备识别身体动作，通过特定动作（如“挥臂挥剑”“下蹲躲避”）实时控制游戏角色，动作数据经 AI 优化后提升交互流畅度。

2 设计目标

经过分析和讨论，我们将影片中展示的场景概括为：利用穿戴设备捕捉动作，并通过模型进行计算后得出对应的信号，利用无线信号发送到另一台嵌入式设备或者终端。如果要复现这一功能，我们的目标是实现两个及以上信息源点对点、无线、近距离、实时连接的需求，这与蓝牙技术的应用场景高度契合。在提供的设备中，我们选用带 BLE 功能的 Nano33 作为硬件基础，依托其低功耗、短距传输优势，让两个信息源无需复杂布线和配置，即可实现数据实时互通，精准匹配近距离点对点下的无线连接需求，并为后续功能开发提供稳定传输支撑。

在实验开始之初，我们计划由一块绑在鞋子上的 Nano 33 采集、推理后通过蓝牙将数据同时发送给终端（电脑）和另一块 Nano 33（接收设备），信号类型采取摩斯电码，含有 dash, dot, space 三种字符串。终端上直接显示发来的信号，接收设备通过连接一枚转子马达将长短信号转化为持续时间不同的振动，接收者就能隐蔽地直接得知发来的信息。

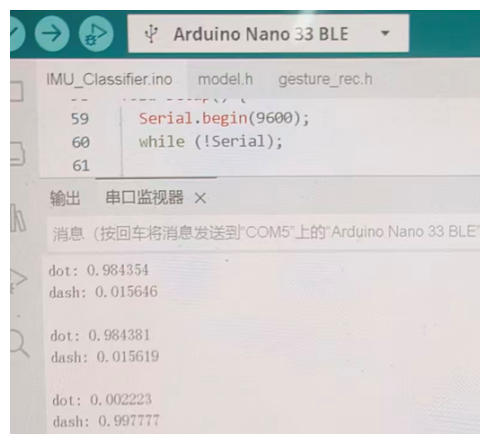
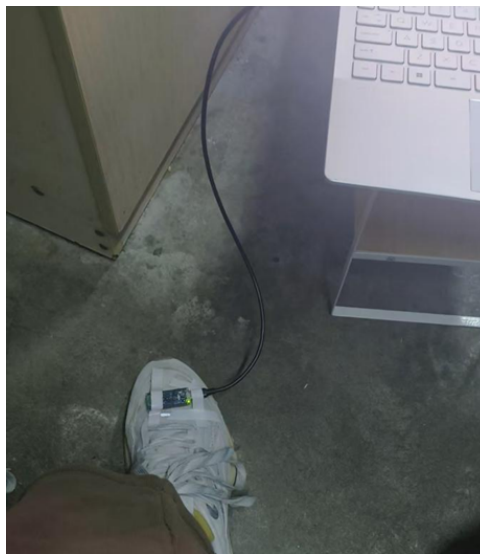
3 实验过程

在这里只介绍组内的设计思路和实践内容，不详细展开具体的实验原理，原理部分详情请见个人报告。

3.1 数据采集与模型训练

利用学在浙大上提供的学习资料，我们进行了学习和进一步开发。首先需要将信号与动作绑定起来，我们定义快速抬脚为 dot, 向右快速摆动为 dash, 向左快速摆动为 space。然后准备开始进行数据的采集、训练：

1. 采集：将 Arduino nano33 固定在鞋背上，连接电脑。采集程序采用条件触发，大幅移动后开始以 119Hz 速率采集 1s 的传感器数据。对于 dot, dash, space 三个动作各采集十次动作数据。为方便记录，在电脑端编写 python 脚本，直接将串口的输出写到 csv 文件中。之后对数据做归一化处理，然后随机打乱，并分成 train, val, test 三部分数据集。



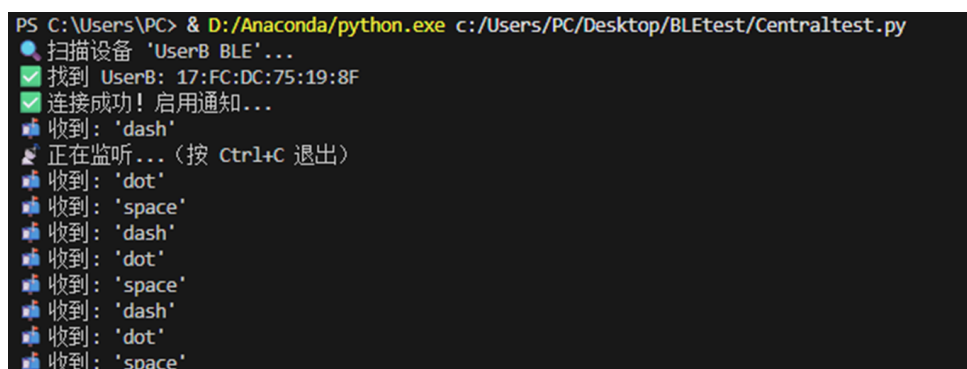
2. 模型：为了保证模型能够嵌入有限存储的单片机又不失性能，我们选择三层全连接层作为隐藏层，神经元个数为 64, 32, 16. 同时选择 cross entropy 作为损失函数。在电脑 CPU 上训练 350 个 epoch。然后把权重参数量化为 uint8 格式，并把模型编码成 Arduino 的头文件格式，可以在 arduino 代码中直接调用。

3. 测试：将 Arduino nano33 以采集数据时的姿态固定在鞋背上。仍然采用条件触发开始收集数据，仍然是 1s 内采集 119 次，随后开始模型推理。测试时会输出对于当前动作的概率判断，在最终代码中将只输出判断的动作类型。

3.2 BLE 设备调试

在 BLE 通信模式中，存在两类设备：BLE 服务端 (BLE Server) 和 BLE 客户端 (BLE Client)。通信时，BLE 服务端向外发送信号，可以被附近的 BLE 客户端发现，一个 BLE 客户端可以连接特定的服务端，然后读取服务端发送的信号数据。在本项目中，发送数据的 Nano 33 是服务端（广播设备），接收的终端是客户端（监听设备）。

经过对学在浙大上的资料学习，我们决定借用提供的思路，重写相关代码，利用 python 脚本在终端进行接收和操作。广播设备在烧录代码并上电后自动开始向周围广播，监听设备通过识别 UUID 与广播设备建立连接，连接成功后广播设备开始传输数据。定义 Notify 服务后，传输的数据会自动在终端上显示。发送开发板命名为 UserB，目标开发板命名为 UserA。首先我们测试连接，UserB 设备不间断地循环发送 dash, dot, space 信号，在终端可以看到信号的显示。



```
PS C:\Users\PC> & D:/Anaconda/python.exe c:/Users/PC/Desktop/BLEtest/Centraltest.py
扫描设备 'UserB BLE'...
找到 UserB: 17:FC:DC:75:19:8F
连接成功! 启用通知...
收到: 'dash'
正在监听... (按 Ctrl+C 退出)
收到: 'dot'
收到: 'space'
收到: 'dash'
收到: 'dot'
收到: 'space'
收到: 'dash'
收到: 'dot'
收到: 'space'
```

考虑到后续的可开发性以及可能的多设备沟通，我们决定略微修改方案，让电脑作为中继设备向其他接收端发送数据。这样，如果有其他人加入通讯或者设备名称改变，又或者要实现所有设备都独立具有接收、发送功能，只需要在电脑终端更改脚本即可，不需要重新对 Arduino 设备进行配置。这样，电脑就像一个交换机，任何一方想要实现对所有人通信或者对指定目标通信都可以通过 python 脚本编程实现，可以大大提高扩展性。如果能够做成客户端，实用性将更上一层楼。

在连接调试成功之后我们将模型烧入开发板，却发现广播设备正常广播，能够被搜索到但不能建立连接。为了找出问题原因，我们进行了单步调试，即在模型每一步骤完成后都通过串口发送一行完成提示。经过调试发现，模型卡在了推理这一步，从这里开始后面的部分都没有正常完成。我们查询了资料、向助教进行了询问，认为这是由于 BLE 的“心跳”机制导致：BLE 每隔一段时间会向监听设备发送确认信号，如果使用了阻塞类型的函数将会阻碍这一过程使得 BLE 失效。在之前的测试中已经发现 delay() 超过 400 毫秒会导致这个问题，所以我们设想可以通过压缩模型缩短运行时间来解决，不

3.3 MQTT 连接调试

MQTT 也具有其独特的优势。为不稳定的网络环境下的资源受限设备，提供高效、可靠的双向通信，同时代码占用量小，用户端可以较为轻松的处理。此外，MQTT 模块的连接稳定，不会存在因网络波动导致信息丢失的问题。在本实验中，可以通过 java 环境搭建一个 MQTT 服务器，并将其与 arduino 模块连接，实现通信功能。我们用一块 ESP32 开发板进行硬件连接与固件烧录，将 wifi 模块配置成需要的模式，并安装 java 环境，配置环境变量。本地环境配置好之后，还需要下载 apollo，随后通过 cmd 创建 mybroker，运行 apollo-broker service。

打开 `apollo`, 将服务器中有关电脑 `ip` 的代码进行修改, 更换为实验电脑的 `ip` 地址, 保存完成后就完成了配置。然后, 下载有关的库, 将原有代码中关于电脑 `ip` 地址和手机热点信息的代码进行修改, 随后运行程序, 得到模块已连接的提示。

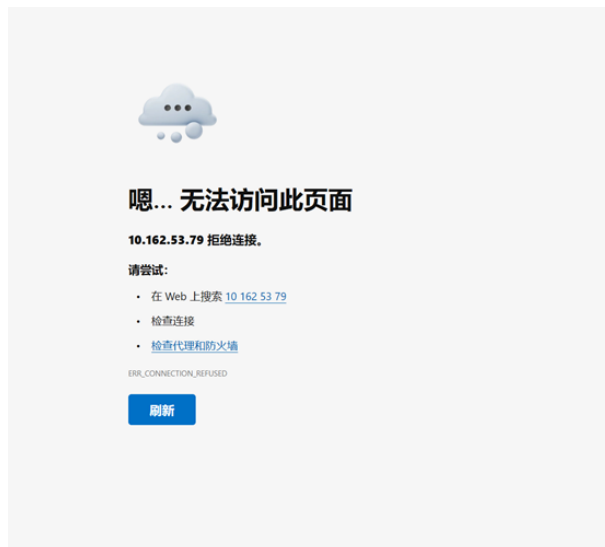
这个过程比较繁琐，也因此带来了诸多问题：

1. 编写 cmd 时程序报错，C 盘运行显示 E 盘出错：推测是下载时操作不当导致一些

```
C:\Users\王舜玺\Downloads\mqtt_wifi\mqtt_wifi\wifi-mqtt-connect\software\apache-apollo-1.7.1-windows-distro2\apache-apol
lo-1.7.1\bin\mybroker\bin>apollo-broker.cmd run
"E:\study\course_EC_kmh\arduino2stu\mqtt_wifi\wifi-mqtt-connect\software\apache-apollo-1.7.1-windows-distro2\apache
-apollo-1.7.1\bin\apollo" 不是内部或外部命令，也不是可运行的程序
或批处理文件。
```

程序丢失，造成 cmd 编写时的错误。图中所示是助教存程序的位置，推测是解压操作有问题，系统判定的位置还在助教那边，没有存放在这台电脑上。

2. 服务器无法正常运行：推测是因为操作频繁，且连接 arduino 时服务器未配置完善，



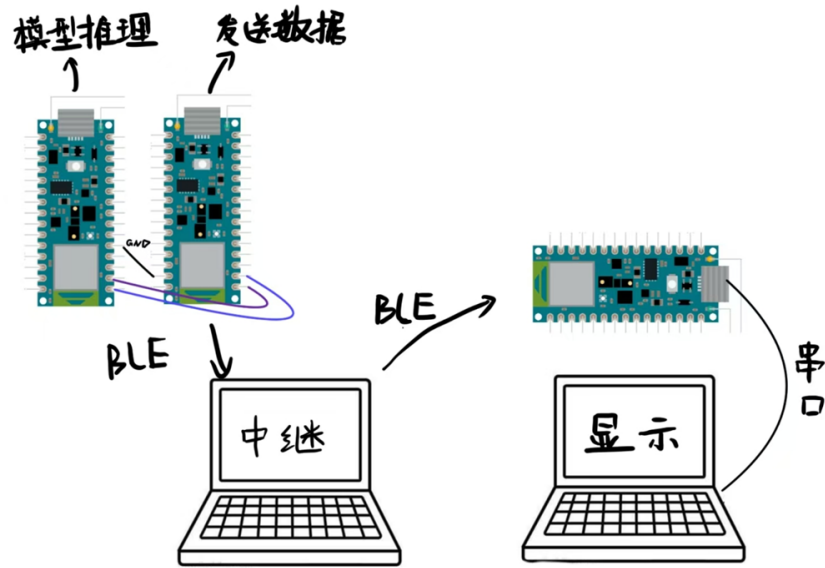
导致系统崩溃，服务器无法正常运行；另外在服务器配置时也有不当操作，由于服务器自身的不稳定，这些操作会导致运行崩溃。

3. ESP32 与服务器连接失效：可能是器件自身存在问题或搭建的服务器有 bug，导致连接后会迅速失效。

由于时间比较紧张，没有办法很仔细地学习、了解 MQTT 的相关知识，在实验过程中出现的诸多问题没法有效解决。我们也向助教寻求了帮助，但是最终无法实现稳定的联通，服务器连接的成功概率不是很高。再加上进一步开发的难度未知，我们只能再次寻找新的思路。

3.4 应急方案与最终实现

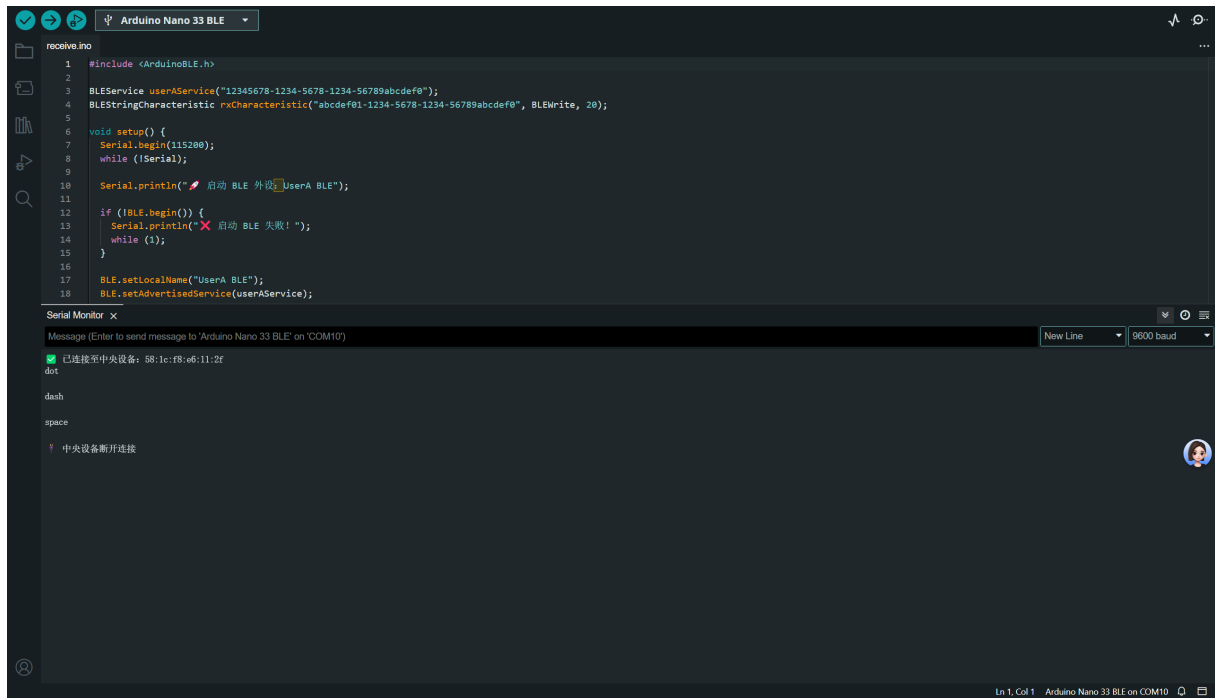
由于我们已经有了充足的 BLE 调试经验，就此放弃有些来不及，也有些可惜。最后我们想出了一个不够实用但一定可以实现完整效果的办法：依旧使用 BLE，将一块开发板专门用于模型推理，另一块开发板专门用于发送数据，两块开发板通过面包板、杜邦线实现串口连接，合起来作为数据的发送端。由于此时时间紧迫，我们舍弃了转子马达的调试，为了展示接收端能够接收到数据，将其与电脑连接，通过串口显示器展示发来的数据。具体示意图如下：



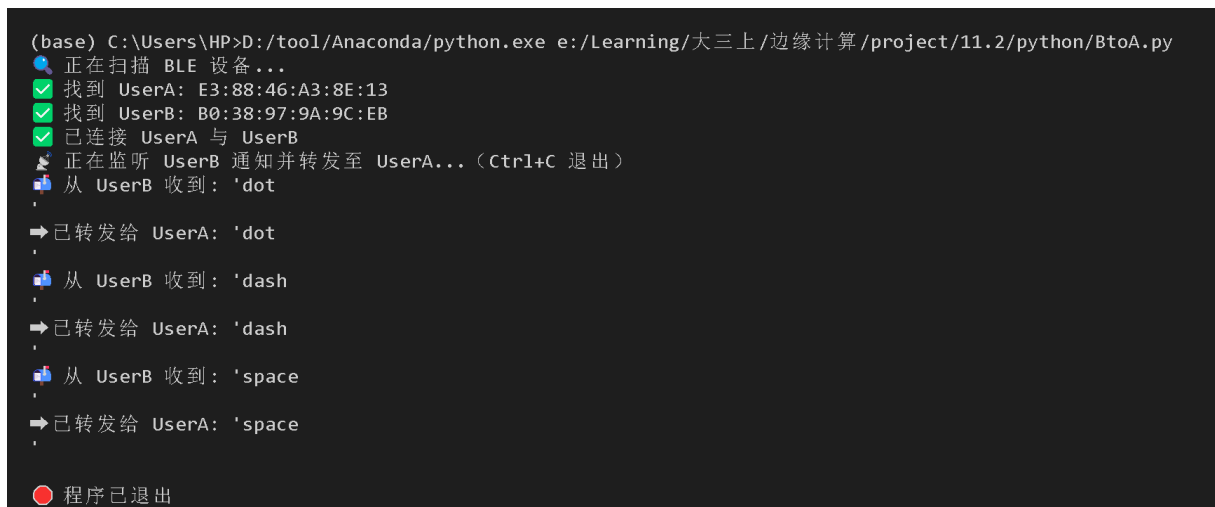
其中两块板子的 TX 与 RX 引脚交叉进行连接。在这里需要将前面写好的代码重新拆分，第一块板子完成模型推理和串口通信，第二块板子完成 BLE 发送。在写串口逻辑时，由于没有注意开发板间通信的函数不是 `Serial()` 而是 `Serial1()`，耽误了一段时间。需要注意的是，前者专门用于与电脑串口监视器的通信。除此之外，还要在接收端也将得到的数据通过串口监视器发送出来。在一切调试好之后，我们将两块板子固定起来绑在鞋上作为发送端：



Nano 33 开发板功耗较低，充电宝供电时识别不到有设备在充电，30 秒后会自动断电。所以在展示时我们将两块开发板连接到了笔记本电脑的 USB 接口上得到稳定的供电。具体结果请见展示视频。最后接收端正常接收数据：



终端显示效果如图所示，数据转发工作正常完成：



4 扩展性与后续开发

本项目具有丰富的扩展性。其中有一些功能是我们设计了但没有实现的，还有一些功能是在我们调试过程中想到的。前文已经提到过多人参与通讯的设想，只不过受限于 BLE 未调试成功等原因没有很好地实现。还有一些可扩展内容如下：

1. 可交互的客户端界面：python 具有很高的集成度，而且开发较为简便。由于用户可能需要修改要搜索的设备名称、添加密钥等，可以制作一个 app 来完成这些交互功能。

2. 接收端的表现形式：除了转子马达之外，还可以通过其他方式进行显示，比如 LED 等。也可以在接收端部署模型，将传过来的数据与某种特定信号一一对应，然后再显示。
3. 终端的再开发：终端具有更强大的性能和更少的推理时间，开发板传来的数据可以实时利用终端的模型进行运算，相当于对数据做二次处理和分析，可以传递更有效的信号。

5 反思与总结

本项目由于种种原因没有达到预期效果，最终完成度在 60% 到 80% 左右。在实践中我们遇到了各种各样的问题，有的是我们自身开发能力不足导致，也有许多问题是鲜有人遇见因而没有办法解决的，甚至有一些现象的机理我们和助教讨论之后仍然无法解释清楚。不过很多在解决这些问题的同时，我们也积累了很多理论知识，比如对蓝牙连接的原理有了更深刻的认识，以及对边缘计算这个功能的优势、劣势都有了更明确的认知。最终，面临技术困难和时间压力，我们为了实现基本效果想出了效率最高的解决策略，对原方案进行了取舍。这某种程度上也说明我们在面对实际工程问题时解决问题的能力得到了锻炼。在组员们密切的配合下，我们迅速地利用这个解决策略达成了目标。